

PROMPT — Conectar db-storage y corregir endpoints grupales

Pegar completo en Replit

Hay un problema crítico que necesito resolver urgente: el sistema tiene implementado `server/db-storage.ts` con queries reales de PostgreSQL pero `server/routes.ts` sigue importando desde `server/storage.ts` (MemStorage). Los datos se pierden con cada reinicio. Necesito hacer el cambio definitivo ahora.

PASO 1 — Cambiar el import (obligatorio, no opcional)

En `server/routes.ts` , en la línea que dice:

```
import { storage } from "../storage";
```

Cambiarla por:

```
import { storage } from "../db-storage";
```

Este es el cambio más importante. Debe hacerse sí o sí.

PASO 2 — Resolver errores de compilación

Después del cambio del import, ejecutar `npm run check` para ver qué métodos usa `routes.ts` que no existen en `db-storage.ts` . Por cada error:

1. Identificar el método faltante
2. Implementarlo en `db-storage.ts` usando el mismo patrón Drizzle que ya tienen los otros métodos

El patrón estándar es:

```

async getNombreItems(): Promise<Tipo[]> {
  return await db.select().from(schema.tabla);
}
async getNombreItem(id: string): Promise<Tipo | undefined> {
  const result = await db.select().from(schema.tabla).where(eq(schema.tabla.id, id));
  return result[0];
}
async createNombreItem(data: InsertTipo): Promise<Tipo> {
  const result = await db.insert(schema.tabla).values({ id: randomUUID(), ...data });
  return result[0];
}
async updateNombreItem(id: string, data: Partial<InsertTipo>): Promise<Tipo | undefined> {
  const result = await db.update(schema.tabla).set(data).where(eq(schema.tabla.id, id));
  return result[0];
}
async deleteNombreItem(id: string): Promise<boolean> {
  const result = await db.delete(schema.tabla).where(eq(schema.tabla.id, id)).returning('*');
  return result.length > 0;
}

```

Repetir hasta que `npm run check` no dé ningún error de TypeScript relacionado con métodos faltantes.

PASO 3 — Ejecutar el seed

Una vez que el servidor compila sin errores, ejecutar el seed para cargar los datos de ejemplo en PostgreSQL:

```
npx tsx server/seed.ts
```

Si el seed falla porque los datos ya existen (constraint de unique), agregar lógica de "upsert" o verificar si ya existe antes de insertar. El seed no debe fallar si se corre más de una vez.

PASO 4 — Corregir endpoints de operaciones grupales

El frontend en `group-detail.tsx` llama a estos endpoints:

- `POST /api/groups/:groupId/check-in-all`
- `POST /api/groups/:groupId/check-out-all`

Estos endpoints no existen en `routes.ts` . Agregarlos con la siguiente lógica:

POST `/api/groups/:groupId/check-in-all`

```
app.post("/api/groups/:groupId/check-in-all", requireAuth, async (req, res) => {
  try {
    const result = await storage.bulkCheckIn(req.params.groupId);
    res.json(result);
  } catch (error) {
    res.status(500).json({ error: "Error en check-in grupal" });
  }
});
```

Implementar el método `bulkCheckIn(groupId: string)` en `db-storage.ts` :

```
async bulkCheckIn(groupId: string): Promise<{ processed: number; skipped: number;
// 1. Obtener links del grupo para encontrar las reservas
const links = await db.select().from(schema.groupReservationLinks)
  .where(eq(schema.groupReservationLinks.groupId, groupId));

let processed = 0;
let skipped = 0;
const skippedRooms: string[] = [];

for (const link of links) {
  // 2. Obtener la reserva
  const [reservation] = await db.select().from(schema.reservations)
    .where(eq(schema.reservations.id, link.reservationId));

  if (!reservation || reservation.status !== "confirmed") continue;

  // 3. Verificar el estado de la habitación
  const [room] = await db.select().from(schema.rooms)
    .where(eq(schema.rooms.id, reservation.roomId));

  if (!room || room.status !== "available") {
    skipped++;
    if (room) skippedRooms.push(room.roomNumber);
    continue;
  }

  // 4. Hacer check-in
  await db.update(schema.reservations)
    .set({ status: "checked_in" })
    .where(eq(schema.reservations.id, reservation.id));
```

```

    await db.update(schema.rooms)
      .set({ status: "occupied" })
      .where(eq(schema.rooms.id, room.id));

    processed++;
  }

  return { processed, skipped, skippedRooms };
}

```

POST /api/groups/:groupId/check-out-all

```

app.post("/api/groups/:groupId/check-out-all", requireAuth, async (req, res) => {
  try {
    const result = await storage.bulkCheckOut(req.params.groupId);
    res.json(result);
  } catch (error) {
    res.status(500).json({ error: "Error en check-out grupal" });
  }
});

```

Implementar el método bulkCheckOut(groupId: string) en db-storage.ts :

```

async bulkCheckOut(groupId: string): Promise<{ processed: number; skipped: number
const links = await db.select().from(schema.groupReservationLinks)
  .where(eq(schema.groupReservationLinks.groupId, groupId));

let processed = 0;
let skipped = 0;
const pendingBalance: Array<{ room: string; guestName: string; balance: number

for (const link of links) {
  const [reservation] = await db.select().from(schema.reservations)
    .where(eq(schema.reservations.id, link.reservationId));

  if (!reservation || reservation.status !== "checked_in") continue;

  // Calcular saldo
  const charges = await db.select().from(schema.charges)
    .where(eq(schema.charges.reservationId, reservation.id));
  const payments = await db.select().from(schema.payments)
    .where(eq(schema.payments.reservationId, reservation.id));

```

```

const totalCharges = charges.reduce((sum, c) => sum + parseFloat(c.amount) || 0);
const totalPayments = payments.reduce((sum, p) => sum + parseFloat(p.amount) || 0);
const balance = totalCharges - totalPayments;

const [room] = await db.select().from(schema.rooms)
  .where(eq(schema.rooms.id, reservation.roomId));

if (balance > 0) {
  skipped++;
  pendingBalance.push({
    room: room?.roomNumber || reservation.roomId,
    guestName: reservation.guestName || "Sin nombre",
    balance
  });
  continue;
}

// Hacer check-out
await db.update(schema.reservations)
  .set({ status: "checked_out" })
  .where(eq(schema.reservations.id, reservation.id));

await db.update(schema.rooms)
  .set({ status: "cleaning" })
  .where(eq(schema.rooms.id, reservation.roomId));

// Crear tarea de housekeeping
const group = await db.select().from(schema.groups)
  .where(eq(schema.groups.id, groupId));

await db.insert(schema.housekeepingTasks).values({
  id: randomUUID(),
  roomId: reservation.roomId,
  type: "checkout_clean",
  priority: "high",
  status: "pending",
  notes: `Check-out grupal - ${group[0]?.name || groupId}`,
  createdAt: new Date().toISOString(),
});

processed++;
}

return { processed, skipped, pendingBalance };
}

```

PASO 5 — Verificación final

Ejecutar estas verificaciones en orden:

1. `npm run check` → debe completar sin errores de TypeScript
2. Iniciar el servidor → debe arrancar sin errores
3. Verificar que `GET /api/rooms` devuelve datos (no array vacío)
4. Verificar que `GET /api/reservations` devuelve datos
5. Verificar que `GET /api/dashboard/stats` responde correctamente
6. Verificar que el login funciona con usuario `admin` y contraseña `maran2026`

Si alguno de estos puntos falla, corregirlo antes de dar la tarea por terminada.

IMPORTANTE

- No crear archivos nuevos innecesarios
- No modificar `server/storage.ts` — ese archivo queda como respaldo pero ya no se usa
- No revertir el import de `db-storage` bajo ninguna circunstancia
- Si un método de `db-storage.ts` no funciona correctamente, corregirlo — no volver a `storage.ts`